



8. Arquitetura Web

SCC0219 – Introdução ao Desenvolvimento Web

Prof^a. Dr^a. Bruna C. Rodrigues da Cunha

brunaru@icmc.usp.br

Service-Oriented Architecture (SOA)

- SOA é uma **arquitetura** para o desenvolvimento de sistemas distribuídos em que os componentes são serviços independentes. Características:
 - **Acoplamento fraco**: uma mudança efetuada em um componente não afeta os outros componentes.
 - **Interoperabilidade**: toda troca de mensagem é independente de plataforma.
 - **Reuso**: os serviços são utilizados por diversos cliente, independentemente.
 - **Capacidade de descoberta**: o ambiente de execução e o contrato de um serviço devem ser identificados e vinculados durante a execução.

Web Services

Um **Web Service** é:

- um componente de software reutilizável, fracamente acoplado, que encapsula funcionalidades discretas, que pode ser distribuído e acessado programaticamente
- uma forma de implementar uma SOA
- acessado usando protocolos padrões da Internet e arquivos (geralmente em XML ou JSON)
- independente da plataforma e da linguagem de programação.



Web Services

- Sua comunicação segue alguns protocolos, principalmente relacionados ao formato da transmissão de dados, o que permite que qualquer outro sistema que siga o mesmo protocolo seja capaz de se comunicar com ele.
- Devido a isso, os Web Services se tornaram extremamente populares, pois acabaram por permitir a comunicação entre plataformas completamente diferentes (Java, C#, Python, JavaScript etc) sem grandes esforços.
- Web Services podem ser implementados utilizando **SOAP** ou **REST**.

Simple Object Access Protocol (SOAP)

- SOAP é uma especificação de protocolo de mensagens para troca de informações estruturadas
- os serviços implementados contam com comunicação baseada em mensagens XML
- Utiliza a WSDL (Web Service Definition Language) para especificar a interface de serviços por descrições em formato XML de Web Services SOAP.

Web Service Description Language (WSDL)

- A interface de um serviço deve ser definida em **WSDL** (Web Service Description Language).
- A especificação WSDL define:
 - quais operações o serviço suporta e o formato das mensagens relacionadas
 - onde o serviço está localizado

WSDL: Exemplo

```

<wsdl:portType name="ICalculator">
  <wsdl:operation name="Add">
    <wsdl:input wsaw:Action="http://Example.org/ICalculator/Add"
      message="tns:ICalculator_Add_InputMessage" />
    <wsdl:output wsaw:Action="http://Example.org/ICalculator/AddResponse"
      message="tns:ICalculator_Add_OutputMessage" />
  </wsdl:operation>

  <wsdl:operation name="Subtract">
    <wsdl:input wsaw:Action="http://Example.org/ICalculator/Subtract"
      message="tns:ICalculator_Subtract_InputMessage" />
    <wsdl:output wsaw:Action="http://Example.org/ICalculator/
SubtractResponse"
      message="tns:ICalculator_Subtract_OutputMessage" />
  </wsdl:operation>
</wsdl:portType>

```

SOAP

- Os XMLs especificados pelo SOAP seguem um padrão definido dentro do protocolo
- Esse padrão serve para que, a partir de um objeto, seja possível serializar o mesmo para XML e, também, desserializá-lo de volta para o formato original
- A serialização de objetos tem como objetivo formar uma mensagem SOAP, composta pelo objeto denominado envelope SOAP, ou **SOAP-ENV**



SOAP

- Dentro desse envelope existem mais dois componentes:
 - o **header SOAP** (cabeçalho), que possui informações de atributos de **metadados** da requisição, por exemplo, IP de origem e dados de autenticação
 - e o **SOAP body**, que possui as informações referentes à requisição, nomes dos **métodos** que deseja se invocar e o **objeto serializado** que será enviado na requisição.

SOAP: Exemplo de Busca de Revista

```
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-
envelope">
  <soap:Header>
  </soap:Header>
  <soap:Body>
    <m:GetRevista xmlns:m="http://www.servicoxyz.com/ws/revista">
      <m:RevistaNome>Veja</m:RevistaNome>
      <m:RevistaAno>1999</m:RevistaAno>
      <m:RevistaMes>5</m:RevistaMes>
    </m:GetRevista>
  </soap:Body>
</soap:Envelope>
```

SOAP

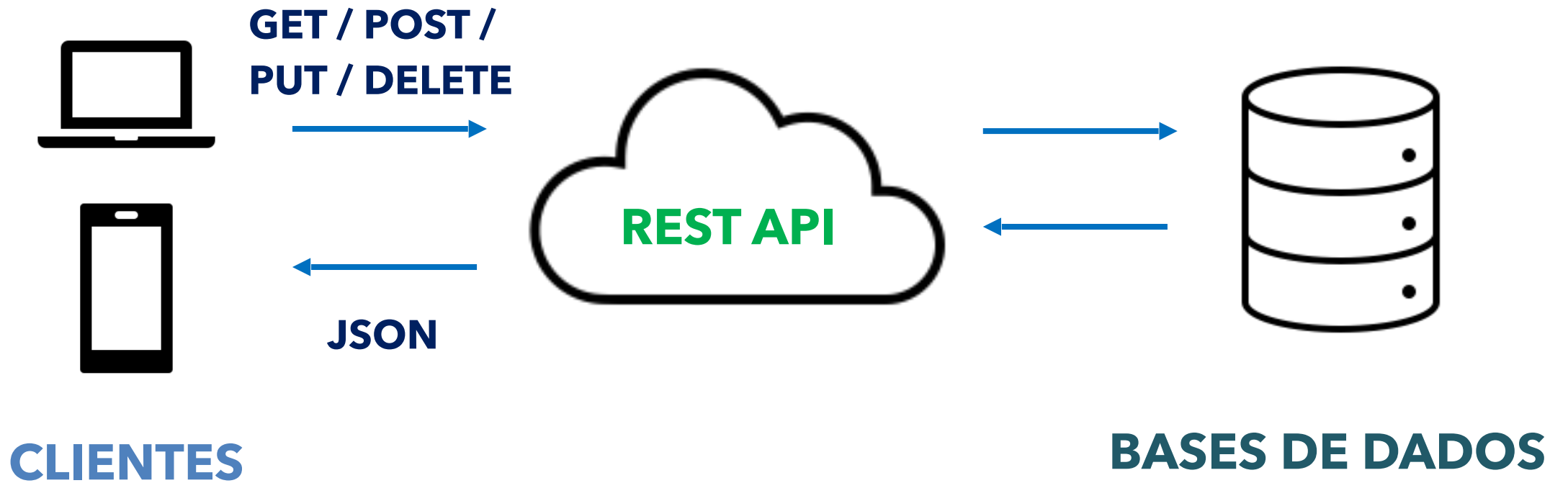
- Chamado de “Big” Web Services
- O problema desse padrão é que ele adiciona um *overhead* considerável por adicionar muitas *tags* de metainformação
- Além disso, a serialização e desserialização das mensagens pode consumir um tempo considerável
- Uma vantagem do SOAP é que ele não limita o protocolo de comunicação utilizado para a troca de mensagens (e.g., HTTP, HTTPS, SMTP, TCP)

Web Services RESTful

- REST (Representational State Transfer) é um estilo arquitetural baseado na transferência de representações de recursos de um servidor para um cliente
- É mais simples que o SOAP/WSDL para implementar serviços Web
- Os serviços REST envolvem uma sobrecarga menor do que os chamados "Big Web Services"
- Proposto por Roy Fielding (2000)

REST

REpresentational State Transfer



REST

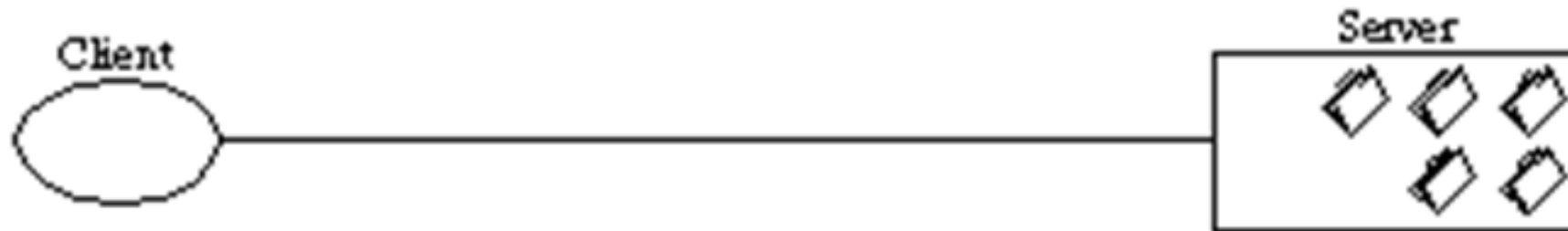
Princípios do estilo arquitetural REST:

1. **Cliente-Servidor**
2. ***Stateless* (Sem Estado)**
3. **Cache**
4. **Sistema em Camadas**
5. **Código sob Demanda**
6. **Interface Uniforme**

REST

1. Cliente-Servidor: cliente e servidor são separados por uma interface uniforme

- O cliente não precisa se preocupar em como os dados são armazenados e manipulados no servidor
- O desenvolvimento entre as partes é independente

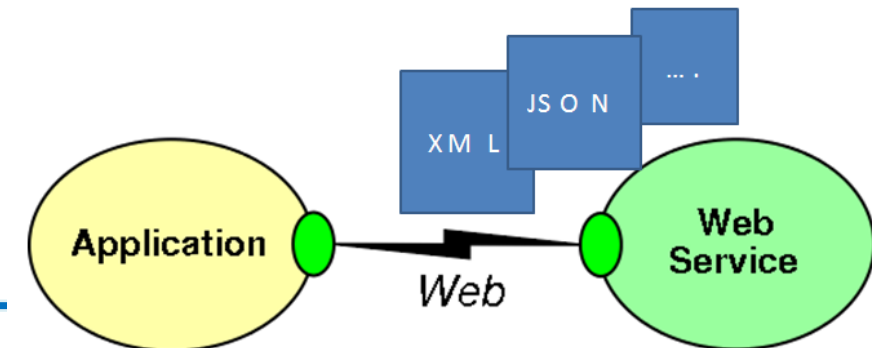


REST

- Mensagens HTTP

- Verbos HTTP: GET, PUT, POST, DELETE
- URI: <http://enderecoweb service/Recurso/{IdRecurso}>
- Exemplos:
 - GET <http://enderecoweb service/Pessoas>
 - GET <http://enderecoweb service/Pessoas/136>

- Dados: JSON ou XML



REST

- URIs não devem conter verbos. Verbos são indicados pelos métodos HTTP.
- Os verbos mais importantes em Web Services são:

Verbo	Operação
GET	Lê recursos
PUT	Modifica ou cria um recurso
DELETE	Apaga recursos
POST	Cria um recurso

Arquitetura REST

- **2. Stateless (Sem Estado):** o servidor não guarda o estado da aplicação para nenhum cliente, cada requisição deve enviar toda informação necessária para seu entendimento. Toda requisição deve ser autossuficiente. Exemplo:

- *Stateful* (Com Estado):

1. GET <http://MyService/Persons/1>
2. GET <http://MyService/NextPerson>



- *Stateless* (Sem Estado):

1. GET <http://MyService/Persons/1>
2. GET <http://MyService/Persons/2>



Arquitetura REST

3. Cache: armazenamento em cache é o conceito de armazenar resultados gerados temporariamente em vez de gera-los novamente para a mesma requisição.

4. Sistema em camadas: a arquitetura do sistema é composta por várias camadas hierárquicas.

5. Código sob Demanda: servidores podem retornar pedaços de código (*scripts*) para serem executados no cliente.

Arquitetura REST

6. Interface Uniforme: a característica central que distingue um sistema que segue o padrão REST é sua ênfase em uma interface uniforme. É definida por 4 restrições de interface:

6.1. Identificação de recursos

6.2. Representação de recursos

6.3. Mensagens auto descritivas

6.4. Hipermídia como o motor do estado do aplicativo

REST Identificação de Recursos

6.1. Identificação de recursos: recursos devem ser acessados por meio de endereços.

- Em Web Services RESTful, recursos são endereçados por URIs
 - URIs não devem conter verbos
 - O nome do recurso DEVE ser no plural
 - As URIs devem ser apresentadas como uma hierarquia de diretórios.
 - Padrão de nomenclatura:

<http://Endereco-URL/NomeServico/NomeRecursos/{IdRecurso}>

Exemplo: <https://icmc.usp.br/ws/students/>

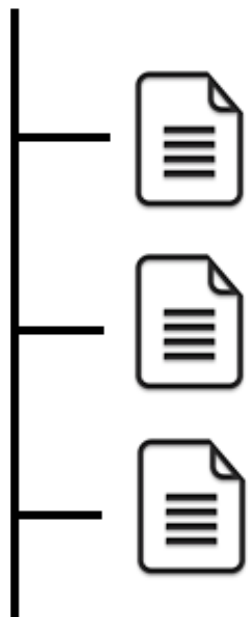
<https://icmc.usp.br/ws/students/56879216>

REST Identificação de Recursos

Analogia com a organização de diretórios...



/pessoas/



alberto.html

carlos.html

maria.html



Web Service

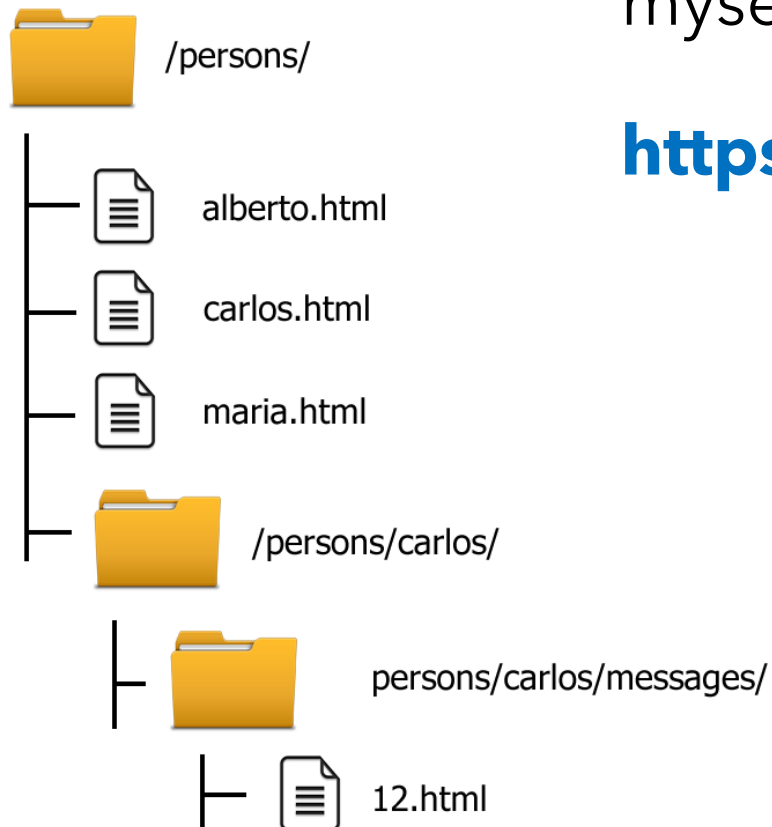
meusite/pessoas/carlos.html

meuservico/pessoas/{id}

Se considerarmos que o id de Carlos é 2,
temos:

<https://myservice/pessoas/2>

REST Identificação de Recursos



myservice/persons/{personId}/messages/{messageId}

<https://myservice/persons/2/messages/12>



Web Service

REST Representação de Recursos

6.2. Representação de recursos: cliente e servidor devem compreender o formato padrão. O formato de representação. JSON e XML são alternativas em WS.

XML

```
<Person>
  <Id>12</Id>
  <Name>Ana Paula Souza</Name>
  <Age>33</Age>
</Person>
```

JSON

```
{
  "Person": {
    "Id": "12",
    "Name": "Ana Paula Souza",
    "Age": "33"
  }
}
```


REST Representação de Recursos

```
{
  "employees": [
    { "firstName": "John", "lastName": "Doe" },
    { "firstName": "Anna", "lastName": "Smith" },
    { "firstName": "Peter", "lastName": "Jones" }
  ]
}
```

```
<employees>
  <employee>
    <firstName>John</firstName> <lastName>Doe</lastName>
  </employee>
  <employee>
    <firstName>Anna</firstName> <lastName>Smith</lastName>
  </employee>
  <employee>
    <firstName>Peter</firstName> <lastName>Jones</lastName>
  </employee>
</employees>
```

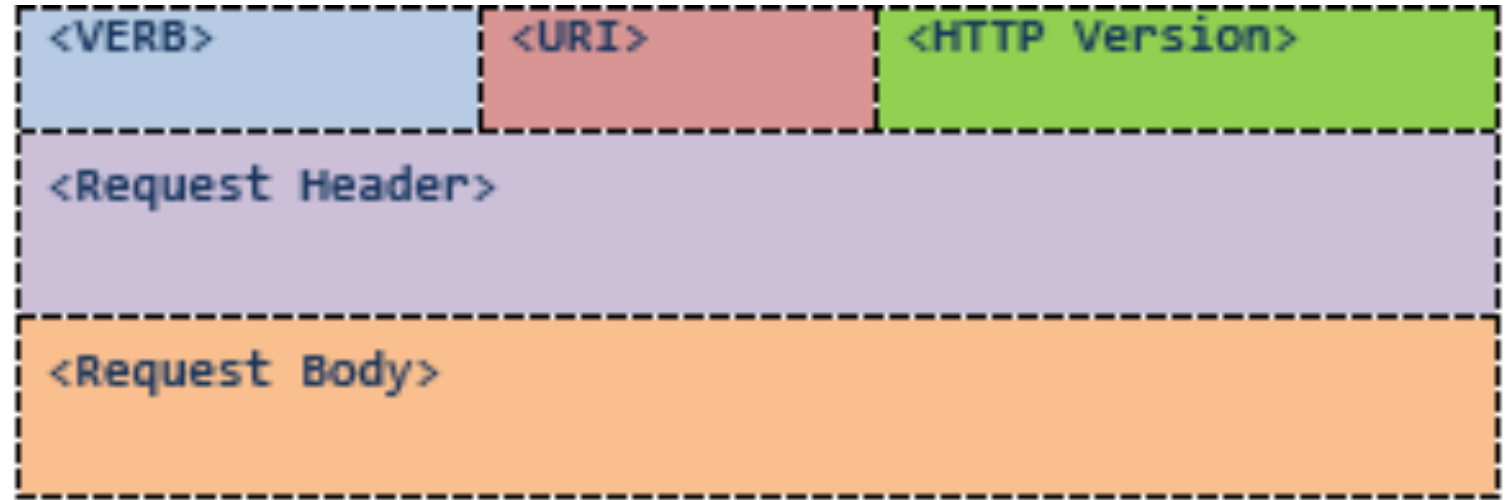
REST Mensagens

6.3. Mensagens auto descritivas: cada requisição do cliente e cada resposta do servidor são uma mensagem

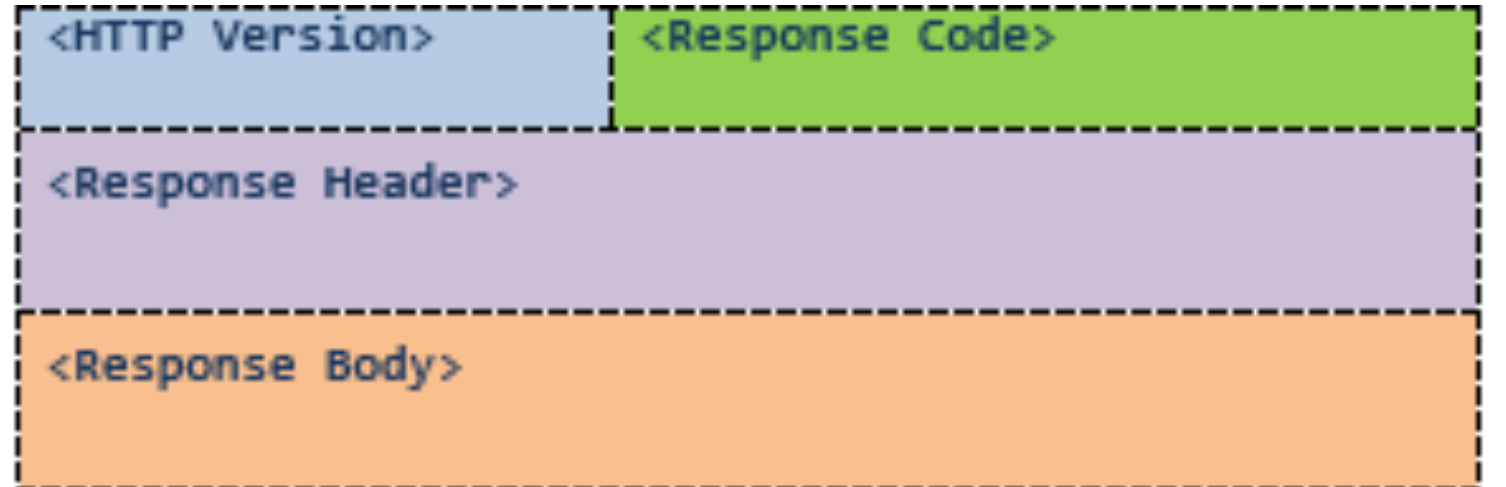
- Toda mensagem deve ser padronizada e conter dados (suficientes) que a descrevem
- Web Services RESTful utilizam mensagens do protocolo HTTP
 - Verbo HTTP: ação a ser realizada
 - URI: recurso acessado
 - Corpo: dados enviados/recebidos
 - Response Code: código de sucesso/erro
 - Cabeçalho: demais informações necessárias (e.g., autenticação)

REST

REQUEST



RESPONSE



REST

6.4. Hipermídia como o motor do estado do aplicativo (HATEOAS):

identificadores devem ser usados dentro de recursos para encontrar outros recursos.

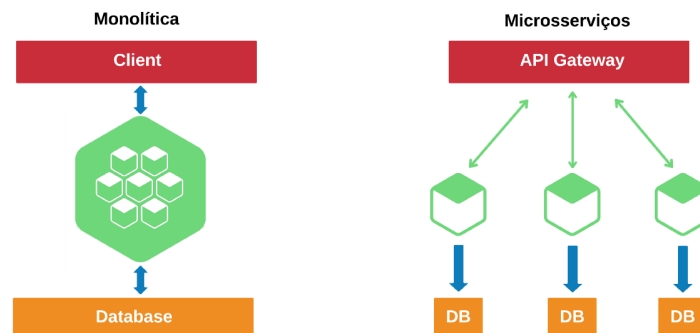
Exemplo: <https://gist.github.com/mellomaths/a27655c3dcae033a5d46b5a2deb8848d>

Web Services RESTful

- **“Big” Web Services:** utilizam mensagens XML que seguem o padrão SOAP e necessitam de um documento XML (WSDL) para descrever a interface dos serviços oferecidos. São robustos, pouco flexíveis e verbosos
- **RESTful Web Services:** desenvolvidos de acordo com os princípios REST e sobre o protocolo HTTP. São mais flexíveis, leves, simples de implementar e utilizar

Microserviços

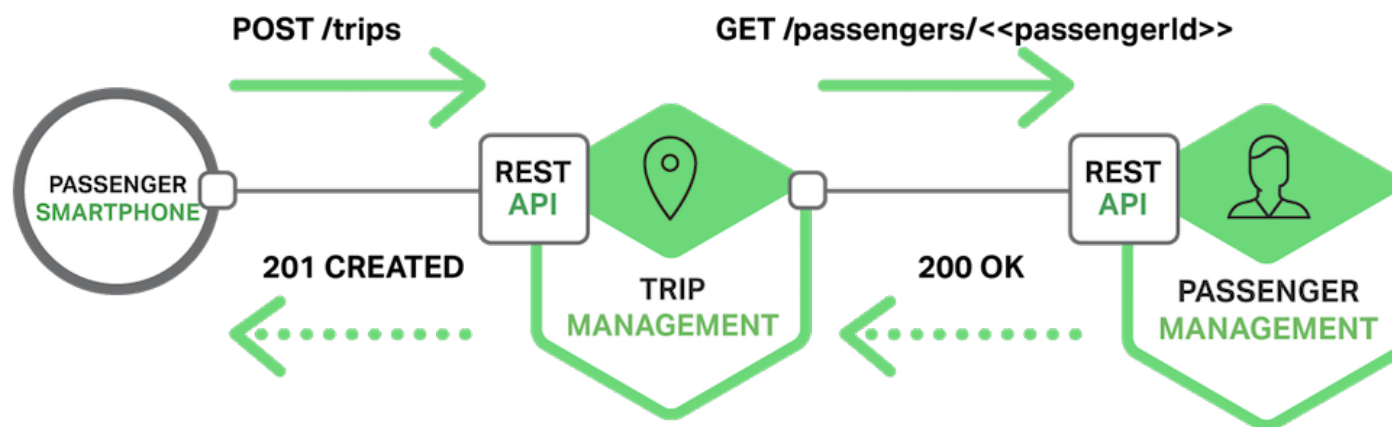
- Microserviços é uma abordagem arquitetônica
- Web Service é uma forma adotada para implementar microserviços
- Nem todo Web Service é microserviço: posso ter um grande Web Service monolítico (centralizado, um único pacote, um único banco)
- Na arquitetura de microserviços o *backend* é fragmentado em várias partes com escopo bem definidos (arquitetura distribuída, pacotes independentes, vários bancos, facilmente descentralizado)



Arquitetura de Microserviços

Microserviços

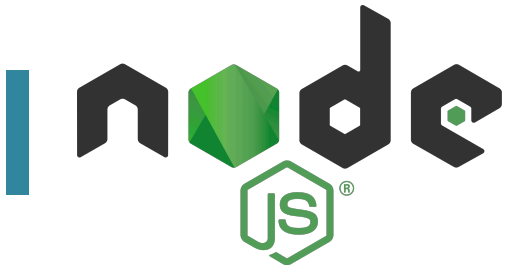
- Podemos implementar um sistema em uma abordagem de microserviços com Web Services no padrão arquitetural REST.
- Observação: a comunicação não precisa ser feita apenas entre cliente e serviços, pode haver comunicação entre serviços.



Microserviços

- Web Service: Componente
- REST: estilo/padrão arquitetural
- Microserviço: Abordagem Arquitetônica (Arquitetura)

- **Vantagens de Microserviços:**
 - Manutenibilidade
 - Escalabilidade
 - Disponibilidade
 - Liberdade tecnológica



- **Node.js** é um ambiente de execução de *software* escrito em JavaScript que permite a criação de aplicações para a web.
- Permite o uso do JavaScript no lado do servidor, tornando-o uma linguagem *full-stack*
- Foi construído sobre o motor JavaScript do Google Chrome (V8) e utiliza um modelo de I/O baseado em eventos não-bloqueantes
- Uma das principais vantagens do Node.js é sua capacidade de lidar com grandes quantidades de requisições devido a sua arquitetura
- Foi criado em 2009

https://nodejs.org/



Node.js® is a JavaScript runtime built on [Chrome's V8 JavaScript engine](#).

Download for macOS (x64)

16.16.0 LTS

Recommended For Most Users

18.7.0 Current

Latest Features

[Other Downloads](#) | [Changelog](#) | [API Docs](#)

[Other Downloads](#) | [Changelog](#) | [API Docs](#)

Or have a look at the [Long Term Support \(LTS\) schedule](#)

NPM

- O NPM (**Node Package Manager**) é uma ferramenta de linha de comando que facilita a instalação e gerenciamento de bibliotecas que podem ser usadas em aplicações Node.js
- O NPM foi criado para ajudar desenvolvedores a compartilhar e reutilizar código e se tornou um dos maiores repositórios de pacotes do mundo
- O NPM também fornece uma série de comandos úteis para executar scripts e tarefas, como *npm start* para iniciar o aplicativo e *npm test* para executar testes. Estes comandos podem ser especificados no arquivo *package.json*.
- Node.js e NPM são ferramentas essenciais para construir aplicações *backend* em JS e gerenciar dependências

NPM: Comandos Essenciais

npm init

- Usado para **inicializar um novo projeto** Node.js e criar um arquivo package.json
- Quando você executa npm init, você será guiado por um processo interativo que lhe pedirá informações sobre o seu projeto, como nome, versão, descrição e licenças
- Ao finalizar, o NPM criará o arquivo **package.json** que contém as informações sobre o seu projeto. Este arquivo é importante por permitir que outros inicializem facilmente o seu projeto

NPM: Comandos Essenciais

npm install nome-do-pacote

- Comando utilizado para instalar pacotes no projeto. O NPM procura o pacote especificado na Internet e o baixa para o diretório do projeto
- O pacote é adicionado à seção **dependencies** do arquivo **package.json**, o que significa que ele será instalado automaticamente quando alguém instalar o projeto
- O comando **npm install** também pode ser usado sem especificar um pacote específico. Nesse caso, o NPM procurará as dependências especificadas no arquivo **package.json** e as instalará
- O comando **npm i** é uma forma abreviada de **npm install**

NPM: Comandos Essenciais

Variações comuns do comando **npm i**:

- **npm i -g <nome-do-pacote>**: instala o pacote globalmente, o que significa que o pacote ficará disponível para todos os projetos em sua máquina, em vez de apenas para o projeto atual.
- **npm i --save-dev <nome-do-pacote>**: instala o pacote e o adiciona à seção devDependencies do arquivo package.json. Isso significa que o pacote será instalado apenas em modo de desenvolvimento, e não em modo de produção.

Ambiente

- **node -v**
- **npm -v**
- **npm install -g n**
- **n latest**
- **npm install -g nodemon**



Axios

- **Axios** é uma biblioteca popular para fazer solicitações “promise-based” em JavaScript, incluindo aplicativos Node.js
- Ela oferece uma interface de programação de aplicativos (API) fácil de usar para realizar solicitações HTTP, como GET, POST, PUT, DELETE
- Além disso, Axios fornece recursos adicionais, como interceptadores de solicitações e respostas, configurações globais e gerenciamento de erros

Axios



- Exemplo API de Previsão do Tempo: <https://open-meteo.com/>
- URL com parâmetros de retorno de temperaturas mínimas e máximas de Capivari - SP:
- https://api.open-meteo.com/v1/forecast?latitude=-23.00&longitude=-47.51&daily=temperature_2m_max,temperature_2m_min&timezone=auto

- **Antes de tudo: Instalar a biblioteca do Axios!**

npm install axios

Axios: Exemplo

```
import axios from "axios"

async function getPokemonsPorTipo(tipo) {
  const response = await axios.get('https://pokeapi.co/api/v2/type/' + tipo)
  const pokemons = response.data.pokemon
  return pokemons
}

const tipo = 'fairy'
getPokemonsPorTipo(tipo).then(function (pokemons) {
  console.log('Pokémons do tipo ' + tipo + ':')
  for (let pk of pokemons) {
    console.log(pk.pokemon.name)
  }
}).catch(error => {
  console.error(error)
})
```

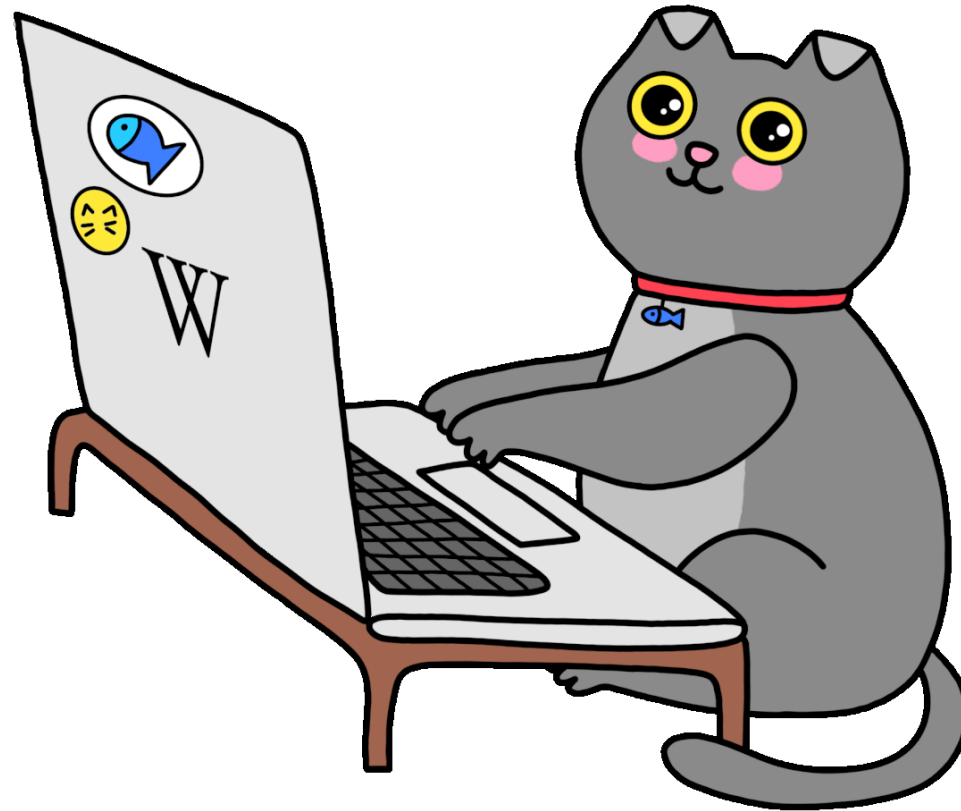
Express

- O Express é um *framework* de aplicações web para o Node.js que fornece uma maneira conveniente e flexível de criar e gerenciar aplicações web
- O Express fornece várias funcionalidades, como gerenciamento de rotas, tratamento de erros, integração com banco de dados e recursos de segurança
- Com ele, desenvolvedores podem se concentrar em seus projetos em vez de se preocuparem com tarefas repetitivas
- Auxilia na criação de APIs REST
- Instalação:

npm i express

Prática

HELLO, WORLD!



Express

```
import express from "express"

const app = express()
const porta = 8080

app.get('/hello', function(request, response) {
  response.send('Olá Mundo!')
})

app.listen(porta, function() {
  console.log('WS rodando na porta ' + porta)
})
```

require vs import

- Em JavaScript, as palavras-chave **require** e **import** são usadas para importar módulos
- O **import** é uma nova palavra-chave usada para importar módulos no ECMAScript 6 (ES6)
- Como se trata de um recurso mais novo e poderoso, vamos preferir usar o **import** à partir de agora

require vs import

Importante!!!

Para a importação com o padrão *import* funcionar, temos de adicionar a seguinte linha em nosso **package.json**:

```
"type": "module",
```

require vs import

```
const express = require('express')
```

=

```
import express from 'express'
```


require vs import

```
const meuModulo = require('meuModulo')
```

é o mesmo que:

```
import * as myModule from ' meuModulo'
```

ou

```
import meuModulo from 'meuModulo'
```

require vs import

Algo bastante útil do **import** é que ele permite importar variáveis e funções específicas, sem precisar trazer um módulo inteiro. Exemplo:

```
// moduloA.js
```

```
export const x = 1
```

```
export const y = 2
```

```
// moduloB.js
```

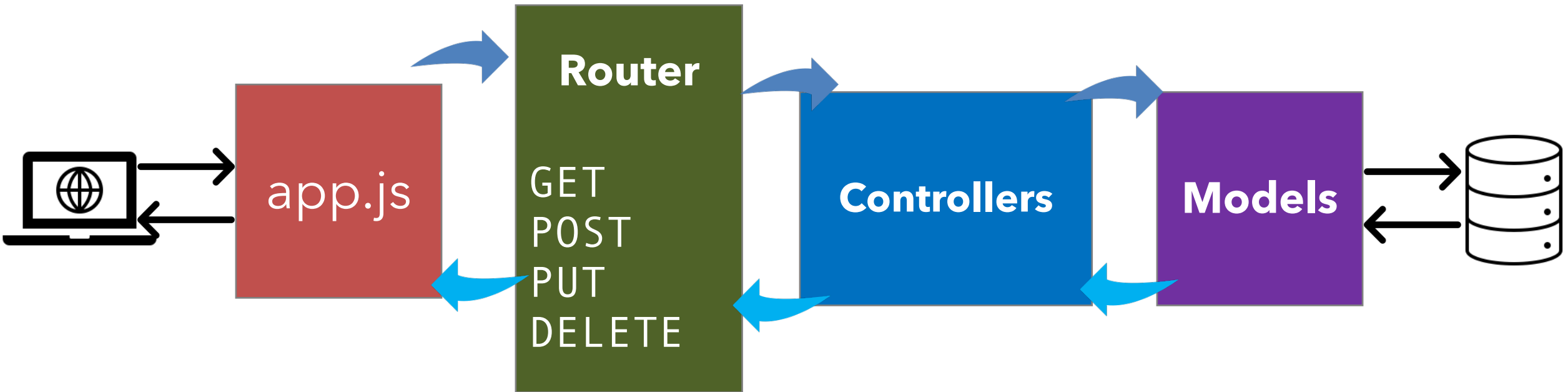
```
import { x, y } from './moduloA.js'
```

Exportar

- Quando você faz seu próprio módulo (um arquivo .js), é preciso indicar quais elementos podem ser importados pelo outros módulos.
- Para isso usa-se a palavra-chave **export**
- Sua variação, **export default**, é usada para indicar o padrão (mínimo) a ser importado. Exemplo:

```
export default {findAll, findByPk, create,  
update, deleteByPk}
```

MVC no Node.js



Model

model/dogbreed.model.js

Responsável por conversar com o banco.

```
class DogBreed {
  constructor(id, breed) {
    this.id = id
    this.breed = breed
  }
}

const breeds = [ new DogBreed(1, "Golden Retriever"), new DogBreed(2, "Bulldog Inglês"), new DogBreed(3,
"Spitz Alemão") new DogBreed(4, "Cheetos") new DogBreed(5, "Poodle") ];

function create(breed) {
  const id = breeds.length + 1;
  breeds.push(new DogBreed(id, breed));
  return findById(id);
}

function findAll() {
  return breeds;
}

function findById(id) {
  const breed = breeds.find(function(breed) { return id == breed.id;});
  return breed;
}

export default { findAll, findById, create }
```

Router

routes/api.routes.js

Indica os caminhos de acesso da API Rest. Exemplo:

```
import express from 'express'
import express from 'express'
import breeds from '../controllers/dogbreed.controller.js'

const routes = express.Router()

router.get("/breeds", breedController.getAll)
router.get("/breeds/:id", breedController.getById)
router.post("/breeds", breedController.create)

export default routes
```

Controller

controllers/dogbreed.controller.js

Responsável por conversar com os módulos de model e responder às requisições adequadamente.

```
import { response } from "express"
import model from "../models/dogbreed.model.js"
export default { getAll, getById, create }

function getAll(request, response) {
  const breeds = model.findAll()
  response.send(breeds)
}
function getById(request, response) {
  const breed = model.findById(request.params.id)
  response.send(breed)
}
function create(request, response) {
  const result = model.create(request.body.breed)
  response.status(201).send(result)
}
```

Ponto de Entrada

app.js

Responsável por “ouvir” uma determinada porta e configurar a aplicação pelo express.

```
import express from 'express'
import routes from './routes/api.routes.js'

const app = express();

app.use(express.json())
app.use(express.urlencoded({extended: true}))
app.use(routes)

app.listen(3000, () => console.log('Servidor iniciado na porta 3000'))
```

| Documentação do Express

Documentação completa da API:

<https://expressjs.com/en/5x/api.html>